Track 1 Team Project

Build Track Capstone

Six Real-World Applications | One Team Choice

React | FastAPI | PostgreSQL | MongoDB | Nginx | GitHub Actions | Git | Linux / Bash

Four weeks of skills come together in one working product. Choose a domain, own the full stack from database to frontend, and ship something real -- clean code, clear documentation, and an app that anyone can run and use.

PACE 2026 | Track 1 | Weeks 1-4 | Team Submission

What This Project Is

This is not a practice exercise. You are building a working product -- something a real user can open, navigate, and use. Pick one domain from the six options. Build the full stack end to end: React frontend, FastAPI backend, PostgreSQL and MongoDB, served through Nginx, with a CI pipeline and proper Git workflow. Every piece must work together. The final submission is a running application, not a collection of disconnected scripts.

Tech Stack -- What Every Project Must Use

React -- Frontend UI | FastAPI -- Backend API | PostgreSQL -- Relational Data | MongoDB -- Document Data | Nginx -- Reverse Proxy | GitHub Actions -- CI Pipeline | Git -- Version Control | Bash -- Setup Script

Output Expectations

Working App
Runs without manual fixes. All screens load. All APIs respond. Both databases connected and functional.

Clean Code
Readable, structured, no dead code. Anyone picking up the repo should understand it without asking.

README
Setup steps, how to run, environment variables, folder structure. Someone new runs it in under 5 minutes.

Arch Diagram
Shows all components, how they connect, and data flow. Any tool or hand-drawn -- must be clear.

Product Doc
Simple, plain language. What the app does, who it is for, what each screen does, what each API does.

Common Requirements -- All Projects

BACKEND -- FASTAPI

- Minimum 8 REST endpoints covering core domain workflows
- Pydantic models for every request and response shape
- HTTPException with clear, meaningful error messages
- PostgreSQL for structured, relational domain data
- MongoDB for flexible, variable, or document-style data
- Swagger UI at /docs -- all endpoints documented and testable
- Minimum 5 pytest tests covering core logic
- Virtual environment + requirements.txt committed

FRONTEND -- REACT

- Minimum 4 screens with React Router navigation
- All data fetched from live FastAPI -- zero hardcoded values
- useState + useEffect pattern for all API calls
- Loading state on every API call, error state on failure
- Service file -- all fetch() calls in one place, not in components
- At least one form that POSTs data to the backend
- Nginx: serves React build as static, proxies /api to FastAPI
- Clean folder structure: pages/, components/, services/

GIT AND DEVOPS

- All work in a team GitHub repo from Day 1
- Feature branches -- no direct commits to main
- Pull requests with descriptions before merging
- Meaningful commit messages -- no "fix", "update", "wip"
- GitHub Actions CI: lint + pytest runs on every push to main
- No secrets, credentials, or .env files committed to repo

DOCUMENTATION AND SETUP

- README: what the app does, full setup guide, how to run
- README covers all environment variables needed
- setup.sh Bash script that installs dependencies on fresh Ubuntu
- Architecture diagram -- all components and connections shown
- Product document -- plain language explanation of the app
- Code comments where logic is non-obvious

Submission Checklist

- [ ] App runs end to end without manual fixes
- [ ] PostgreSQL schema has correct tables and relationships
- [ ] All React screens connected to live API -- no hardcoded data
- [ ] GitHub Actions CI passes on main branch
- [ ] Minimum 5 pytest tests passing
- [ ] Product document submitted
- [ ] README allows a fresh setup in under 5 minutes
- [ ] MongoDB stores appropriate flexible or variable data
- [ ] Swagger UI shows all endpoints with working test inputs
- [ ] No .env or secrets anywhere in the repo
- [ ] Architecture diagram submitted
- [ ] Git history shows real feature branches and PRs

01 PropertyNest -- Real Estate Listing and Rental Management Platform

A platform where property owners list rentals, applicants apply, and lease agreements are managed end to end

WHAT YOU ARE BUILDING

A property rental platform where owners list their properties, prospective tenants browse and filter listings, submit rental applications, and owners review and approve them. Once approved, a lease agreement is created. Tenants can raise maintenance requests and owners can track them.

TECH STACK

PostgreSQL	properties, owners, tenants, applications, leases, maintenance_requests
MongoDB	property descriptions, amenity lists, inspection reports (variable per property)
FastAPI	REST API for all domain workflows -- listings, applications, leases, maintenance
React	Property search, detail view, application form, owner dashboard
Nginx	Serves React build, proxies /api/* to FastAPI on port 8000
GitHub Actions	flake8 lint + pytest on push to main
Bash	setup.sh -- installs Python deps, Node deps, starts Nginx

REACT SCREENS

- Property listing page with search and filters (city, type, rent range)
- Property detail page -- specs + MongoDB description + Apply button
- Rental application form -- applicant details, employment, references
- Owner dashboard -- manage listings, review applications, lease status, maintenance list

REQUIRED API ENDPOINTS

- GET /properties -- list with filters: city, type, min_rent, max_rent, available
- GET /properties/{id} -- property detail, merges PG data with MongoDB description
- POST /properties -- create new listing (owner authenticated by owner_id)
- PATCH /properties/{id} -- update listing details or availability
- POST /applications -- submit rental application with applicant details
- GET /applications?property_id= -- all applications for a property
- PATCH /applications/{id}/status -- approve or reject with reason
- POST /leases -- create lease once application approved
- GET /leases/{id} -- lease detail with tenant and property info
- POST /maintenance -- tenant raises a maintenance request
- GET /maintenance?property_id= -- all requests for a property with status
- PATCH /maintenance/{id}/status -- owner updates request status

NUMBERED REQUIREMENTS

1. Property search must filter by at least 3 parameters using query params
2. Property detail page makes two data calls -- PostgreSQL (specs) + MongoDB (description)
3. Application submission POSTs to backend and shows success confirmation
4. Owner dashboard fetches and lists applications with approve/reject action
5. Approving an application must create a lease record in PostgreSQL automatically
6. Maintenance requests are linked to both property and tenant by foreign key
7. All fetch() calls live in a service file -- not inside components
8. Loading and error states shown on every API call in the UI

02 FleetOps -- Vehicle Fleet and Maintenance Management System

Manage vehicles, assign drivers, track trips, and schedule maintenance across an operational fleet

WHAT YOU ARE BUILDING

A fleet management platform for a transport company. Fleet managers register vehicles and drivers, assign vehicles to drivers, log trips with origin and destination, and schedule maintenance. The system tracks vehicle status (available, on-trip, in-maintenance) and gives managers visibility into utilisation and cost.

TECH STACK

PostgreSQL	vehicles, drivers, trips, assignments, maintenance_schedules, zones
MongoDB	trip logs per journey (stops, notes, fuel readings), maintenance reports (findings, parts replaced)
FastAPI	REST API for vehicles, drivers, trip lifecycle, maintenance management
React	Fleet dashboard, vehicle detail, trip management, maintenance calendar
Nginx	Serves React build, proxies /api/* to FastAPI on port 8000
GitHub Actions	flake8 lint + pytest on push to main
Bash	setup.sh -- installs all deps, seeds initial vehicle and zone data

REACT SCREENS

- Fleet dashboard -- all vehicles with status indicators (available / on-trip / in-maintenance)
- Vehicle detail -- specs, assigned driver, trip history, next maintenance
- Trip management -- start trip, complete trip, view active trips
- Maintenance calendar -- upcoming schedules, overdue alerts, cost history

REQUIRED API ENDPOINTS

- GET /vehicles -- list with status filter (available, on-trip, maintenance)
- GET /vehicles/{id} -- detail with current driver and last maintenance
- POST /vehicles -- register new vehicle with zone assignment
- PATCH /vehicles/{id}/status -- update vehicle status
- POST /drivers -- register driver with license and contact
- POST /trips -- start a trip, sets vehicle status to on-trip
- PATCH /trips/{id}/complete -- end trip, sets vehicle back to available
- GET /trips?vehicle_id= -- trip history for a vehicle
- POST /maintenance -- schedule maintenance for a vehicle
- GET /maintenance?vehicle_id= -- maintenance history with status
- PATCH /maintenance/{id}/complete -- mark done, store report in MongoDB
- GET /fleet/summary -- count by status, trips today, overdue maintenance

NUMBERED REQUIREMENTS

1. Starting a trip must update vehicle status to on-trip in PostgreSQL
2. Completing a trip must update vehicle status back to available
3. Maintenance completion stores the report (findings, parts) in MongoDB
4. Fleet dashboard shows real vehicle status fetched live from API
5. Vehicle detail page makes multiple API calls -- vehicle info + trip history
6. Fleet summary endpoint uses a PostgreSQL aggregation query (GROUP BY status)
7. All fetch() calls live in a service file -- not inside components
8. Loading and error states shown on every API call in the UI

03 EventPulse -- Conference and Multi-Session Event Management Portal

Organise events with multiple sessions and speakers, manage registrations, and collect attendee feedback

WHAT YOU ARE BUILDING

An event management portal for conferences and multi-session events. Organisers create events and schedule sessions with speakers. Attendees browse and register for events, check in on the day, and submit feedback per session. Organisers see registration counts, attendance rates, and feedback summaries.

TECH STACK

PostgreSQL	events, sessions, speakers, registrations, organisers, checkins
MongoDB	speaker profiles (bio, past talks, links), session feedback (variable comments and ratings per session)
FastAPI	REST API for event CRUD, session management, registration, check-in, feedback
React	Event listing, event detail with sessions, registration form, organiser dashboard
Nginx	Serves React build, proxies /api/* to FastAPI on port 8000
GitHub Actions	flake8 lint + pytest on push to main
Bash	setup.sh -- installs all deps, seeds sample events and speakers

REACT SCREENS

- Event listing page -- browse upcoming events with date and capacity
- Event detail page -- overview, full session schedule, speaker cards
- Registration form -- attendee details, ticket type selection, confirmation
- Organiser dashboard -- registration counts, check-in status, feedback summary

REQUIRED API ENDPOINTS

- GET /events -- list with date range and type filter
- GET /events/{id} -- event detail with full session schedule
- POST /events -- create new event with organiser
- POST /sessions -- add session to event with speaker and timeslot
- GET /sessions/{id} -- session detail with speaker profile from MongoDB
- POST /registrations -- register attendee for event
- GET /registrations?event_id= -- all registrations for an event
- POST /checkin/{registration_id} -- mark attendee as checked in
- POST /sessions/{id}/feedback -- submit feedback, stored in MongoDB
- GET /sessions/{id}/feedback -- all feedback for a session from MongoDB
- GET /events/{id}/stats -- registration count, checked-in count, capacity remaining
- GET /speakers -- list all speakers with session count

NUMBERED REQUIREMENTS

1. Registration must check capacity before confirming -- reject if full
2. Check-in must verify registration exists and is not already checked in
3. Session detail fetches speaker profile from MongoDB by speaker_id
4. Feedback is stored in MongoDB with session_id, rating, and comment
5. Event stats endpoint uses a PostgreSQL COUNT query with GROUP BY
6. Event detail page renders multiple sessions dynamically from API using .map()
7. All fetch() calls live in a service file -- not inside components
8. Loading and error states shown on every API call in the UI

04 DeliverIQ -- Order and Last-Mile Delivery Management System

Place orders, assign delivery agents, update status at each stage, and track delivery from placement to doorstep

WHAT YOU ARE BUILDING

A delivery management system for a logistics or food delivery business. Customers place orders, operations assigns delivery agents based on zone, and agents update status at each stage (accepted, picked up, in transit, delivered). Customers and managers see a live timeline of every status change. Completed deliveries collect customer ratings.

TECH STACK

PostgreSQL	orders, customers, agents, zones, assignments, order_items
MongoDB	delivery timeline events per order (status, timestamp, note), customer feedback per delivery
FastAPI	REST API for orders, assignment, status updates, feedback, analytics
React	Place order form, order tracker, agent dashboard, operations view
Nginx	Serves React build, proxies /api/* to FastAPI on port 8000
GitHub Actions	flake8 lint + pytest on push to main
Bash	setup.sh -- installs all deps, seeds zones and agents

REACT SCREENS

- Order placement form -- items, quantity, delivery address, customer info
- Order tracker -- current status with full timeline of events from MongoDB
- Agent dashboard -- assigned deliveries, update status, mark delivered
- Operations view -- all active orders, unassigned queue, assign agent action

REQUIRED API ENDPOINTS

- POST /orders -- place new order with items list and delivery address
- GET /orders/{id} -- order detail with full timeline from MongoDB
- GET /orders?status=&zone_id= -- filter orders by status or zone
- POST /orders/{id}/assign -- assign delivery agent to order
- PATCH /orders/{id}/status -- update delivery status, appends event to MongoDB
- GET /agents -- list agents with zone and active assignment count
- GET /agents/{id}/orders -- all deliveries assigned to an agent
- POST /feedback -- submit rating and comment after delivery (MongoDB)
- GET /orders/{id}/feedback -- fetch feedback for a delivered order
- GET /analytics/summary -- orders by status today using PostgreSQL aggregation
- POST /agents -- register new delivery agent with zone
- GET /zones -- list all delivery zones with agent count

NUMBERED REQUIREMENTS

1. Each status update appends a new event document to MongoDB timeline -- never overwrites
2. Order tracker page renders the MongoDB timeline as a sequential status list
3. Assign endpoint must check agent is in same zone as delivery address
4. Feedback can only be submitted once per order and only after delivered status
5. Analytics summary uses PostgreSQL GROUP BY status with COUNT
6. Operations view shows unassigned orders separately from active ones
7. All fetch() calls live in a service file -- not inside components
8. Loading and error states shown on every API call in the UI

05 CoachZone -- Sports Club Player and Match Management Platform

Register clubs, manage players and coaches, schedule fixtures, submit results, and track team standings

WHAT YOU ARE BUILDING

A sports management platform for a football or cricket league. Club admins register their clubs and teams. Players and coaches are added with their profiles. The league schedules fixtures between teams. After each match, results are submitted with scorelines and scorers. The platform maintains a live league standings table updated from real match data.

TECH STACK

PostgreSQL	clubs, teams, players, coaches, fixtures, match_results, standings
MongoDB	match reports (narrative, highlights, submitted by coach), player performance per season (goals, assists, cards -- variable per sport)
FastAPI	REST API for clubs, players, fixtures, results, standings, reports
React	Club/team directory, player profile, fixture list, live standings table
Nginx	Serves React build, proxies /api/* to FastAPI on port 8000
GitHub Actions	flake8 lint + pytest on push to main
Bash	setup.sh -- installs all deps, seeds league, clubs, and initial fixtures

REACT SCREENS

- Club and team directory -- browse clubs, expand to see team roster and coach
- Player profile -- career stats, current team, performance from MongoDB
- Fixture list -- upcoming and completed matches with scores
- Live standings table -- points, wins, draws, losses, goal difference, sorted by points

REQUIRED API ENDPOINTS

- GET /clubs -- list all clubs with team count
- GET /clubs/{id}/teams -- teams for a club with player count
- POST /players -- register player with team assignment
- GET /players/{id} -- profile merged with MongoDB performance data
- GET /players?team_id= -- all players for a team
- POST /fixtures -- schedule match between two teams with venue and date
- GET /fixtures?team_id=&status= -- fixtures filtered by team or status
- POST /fixtures/{id}/result -- submit match result with scoreline
- GET /fixtures/{id} -- match detail with result and MongoDB report
- POST /fixtures/{id}/report -- submit match report to MongoDB
- GET /standings -- full league table computed from match results
- GET /coaches -- list coaches with team and club info

NUMBERED REQUIREMENTS

1. Submitting a result must update the standings table in PostgreSQL for both teams
2. Standings endpoint computes points dynamically from match results using SQL aggregation
3. Player profile page fetches PostgreSQL data and MongoDB performance in one combined view
4. Fixture list separates upcoming and completed matches using status filter
5. Match report is stored in MongoDB with fixture_id, narrative, and submitted_by
6. A fixture cannot have a result submitted twice -- return error if already recorded
7. All fetch() calls live in a service file -- not inside components
8. Loading and error states shown on every API call in the UI

06 ScholarTrack -- Scholarship Application and Review Portal

Discover scholarships, submit applications with essays, track review decisions through a structured evaluation process

WHAT YOU ARE BUILDING

A scholarship management platform for an institution or foundation. Students browse available scholarships filtered by field and eligibility. They submit applications with an essay and supporting details. Reviewers are assigned applications, score them, and record decisions. Students track their application status and see reviewer feedback once a decision is made.

TECH STACK

PostgreSQL	scholarships, students, applications, reviewers, decisions
MongoDB	application essays and supporting content (variable length per student), reviewer notes and scoring rationale
FastAPI	REST API for scholarships, student profiles, applications, reviews, decisions
React	Scholarship search, application form, status tracker, reviewer panel
Nginx	Serves React build, proxies /api/* to FastAPI on port 8000
GitHub Actions	flake8 lint + pytest on push to main
Bash	setup.sh -- installs all deps, seeds scholarships and reviewer accounts

REACT SCREENS

- Scholarship discovery page -- browse and filter by field, amount, deadline
- Application form -- student details, essay input, scholarship selection
- Application status tracker -- current status, reviewer notes once decision made
- Reviewer panel -- assigned applications, scoring form, decision recording

REQUIRED API ENDPOINTS

- GET /scholarships -- list with filters: field, min_amount, deadline before date
- GET /scholarships/{id} -- detail with eligibility requirements
- POST /students -- create student profile with academic details
- GET /students/{id} -- student profile with application count
- POST /applications -- submit application, essay saved to MongoDB
- GET /applications/{id} -- status from PostgreSQL + essay + reviewer notes from MongoDB
- GET /applications?student_id= -- all applications by a student
- GET /applications?reviewer_id= -- all applications assigned to a reviewer
- POST /applications/{id}/assign -- assign reviewer to application
- POST /applications/{id}/review -- reviewer scores and submits notes to MongoDB
- PATCH /applications/{id}/decision -- record final decision (awarded, shortlisted, rejected)
- GET /scholarships/{id}/stats -- total applied, shortlisted, awarded counts

NUMBERED REQUIREMENTS

1. Application essay is stored in MongoDB -- PostgreSQL stores only the application record and status
2. Application detail endpoint merges PostgreSQL status with MongoDB essay and reviewer notes
3. A student cannot apply to the same scholarship twice -- return error on duplicate
4. Decision can only be recorded after a review score exists for the application
5. Scholarship stats endpoint uses PostgreSQL COUNT and GROUP BY decision
6. Application status tracker shows notes from MongoDB only after decision is recorded
7. All fetch() calls live in a service file -- not inside components
8. Loading and error states shown on every API call in the UI